Credit Card Fraud Detection: Reproduction and Critical Evaluation of an Autoencoder-Based Approach

Final Project - Data Science in Cyber (Dr. Uri Itai)

Source under evaluation: Venelin Valkov, "Credit Card Fraud Detection using Autoencoders in Keras - TensorFlow for Hackers (Part VII)" (blog, GitHub)

Dataset: Kaggle Credit Card Fraud Detection (mlg-ulb/creditcardfraud)

1. Summary of the Source

The problem

The source addresses **credit card fraud detection** under extreme class imbalance: fraudulent transactions are rare (order of 0.1-0.2% of all transactions) and constantly evolving, making it hard to obtain enough labeled fraud examples to train a conventional supervised classifier with confidence. The author frames this as motivation for an **unsupervised anomaly-detection** approach: instead of learning to recognize fraud directly, learn what a normal transaction looks like and flag anything that reconstructs poorly as anomalous.

Why it matters

The author cites global card fraud losses (~$21.8 billion in 2015) as the practical stakes. Beyond the dollar figure, the underlying data-science challenge - severe class imbalance, a moving target (fraud patterns change over time), and asymmetric error costs (missed fraud vs. blocked legitimate purchases) - generalizes to intrusion detection, phishing detection, and other cybersecurity anomaly-detection problems, which is why this is a useful case study beyond fraud specifically.

Proposed solution

A deep **autoencoder** (four dense layers: 14 -> 7 -> 7 -> 29 neurons, tanh/relu activations, L1 regularization on the encoder, mean-squared-error loss, Adam optimizer, 100 epochs, batch size 32) is trained **only on normal transactions**. At inference time, the reconstruction error (MSE between input and output) is computed for every transaction; a fixed threshold (2.9 in the source) separates normal (low error) from fraud (high error) predictions.

Dataset

Kaggle's Credit Card Fraud Detection dataset: 284,807 transactions from European cardholders over two days in September 2013, with 492 labeled frauds (0.172%). Features V1-V28 are PCA components (anonymized for privacy); Time (seconds since first transaction) and Amount are the only untransformed features.

Model / methodology employed

80/20 train/test split, with the training set filtered to normal transactions only so the model never sees fraud during training. The test set retains the natural class mix. Performance is reported via ROC-AUC (0.94) and a confusion matrix at the chosen threshold.

2. Critical Evaluation

The author's main claims

- An autoencoder trained exclusively on normal transactions can separate fraud from normal transactions via reconstruction error, achieving ROC-AUC = 0.94.
- A reconstruction-error threshold of 2.9, chosen by visual inspection of the error distribution on the test set, gives an acceptable operating point.
- The high false-positive rate at that threshold is an acceptable trade-off, to be tuned further based on business requirements.

Are the claims supported by the evidence presented?

Partially. The ROC-AUC claim is a threshold-independent, defensible summary statistic, and our reproduction (Section 5 / notebook) confirms a very similar figure (ROC-AUC ~ 0.946 for the autoencoder, close to the source's 0.94, despite using a slightly different feature/library setup). However, the **evaluation methodology has two material gaps:**

Gap 1 - Threshold selection is a form of test-set leakage

The source selects its operating threshold by inspecting the reconstruction-error histogram **on the test set** - the same data the model is then scored against. This is methodologically unsound: any threshold or hyperparameter chosen by looking at labels/distributions in the evaluation set risks overfitting the reported operating-point metrics (precision, recall, F1) to that specific sample, even if it does not affect threshold-independent metrics like ROC-AUC/PR-AUC.

Our test of this claim: we reproduced the source's exact procedure (pick the F1-maximizing threshold on the full test set, then score on that same set) and compared it against a leakage-free alternative (split the test set into a validation half used only to pick the threshold, and a held-out half used only to score it). Results:

Metric	Leaked threshold (source's method)	Clean threshold (validation-based)
Threshold	7.87	7.87
Precision	0.421	0.435
Recall	0.571	0.585
F1	0.484	0.499
MCC	0.485	0.500

In this specific reproduction, the leakage did not inflate the reported numbers - if anything the clean threshold scored marginally *better*, most likely because the test set (56,746+ rows, including 473 frauds after de-duplication) is large enough that a threshold fit on half of it generalizes well to the other half. **This is an important, honest finding: the leakage risk is real and the practice remains poor methodology in general (it would not be safe to assume this holds on a smaller test set, a different dataset, or under distribution shift), but in this particular case it did not measurably distort the source's headline conclusion.** A reviewer should not take this as a blanket defense of the practice - we would flag it in any resubmission of this methodology regardless of the (here, benign) outcome.

Gap 2 - Missing imbalance-aware metrics

The source reports only ROC-AUC. Under 0.17% class imbalance, ROC-AUC is known to be optimistic because the false-positive rate axis is dominated by the (huge) negative class - a large absolute number of false positives can still look small as a *rate*. We added Precision-Recall AUC and Matthews Correlation Coefficient (MCC), which are more informative under this imbalance. Our autoencoder reproduction scores PR-AUC ~ 0.42 and MCC ~ 0.49-0.50 - figures that look far less impressive than the 0.94 ROC-AUC alone and would have changed the tenor of the source's conclusion if reported.

Is the evaluation methodology appropriate?

Partially appropriate: the normal-only training design is a reasonable response to class imbalance, and ROC-AUC is a legitimate metric to report. But reporting only one metric, and selecting the threshold on the same data used for evaluation, are both avoidable methodological weaknesses for a tutorial whose stated goal is to demonstrate a production-viable detection approach.

Weaknesses / limitations

- No comparison to a supervised baseline - our reproduction shows a simple Random Forest (class_weight="balanced") substantially outperforms the autoencoder on every threshold-dependent metric (F1 0.822 vs. 0.484, MCC 0.833 vs. 0.485) - see Section 5.
- No cross-validation or repeated-split analysis; results rest on a single train/test split.
- No PR-AUC or MCC reported, despite severe imbalance.
- Threshold chosen via visual inspection of the test-set distribution rather than a held-out validation split.

Are the conclusions justified?

The core conclusion - that an autoencoder trained on normal transactions only can meaningfully separate fraud via reconstruction error - is supported and reproducible. The implicit conclusion that this is a *strong* or *production-ready* approach is **not well supported**: once precision/recall/MCC/PR-AUC are examined and a simple supervised baseline is added, the autoencoder is clearly the weakest of the three models we tested on this dataset. We do not think the source's numeric claims are wrong, but the framing ("autoencoders are a mystical and magical thing", per the source) overstates the approach's practical advantage over much simpler supervised alternatives when labels are, as they are here, actually available.

3. Feature Engineering Analysis

Was feature engineering performed, and what features were used?

The source uses the 29 published features as-is (V1-V28 + Amount), dropping Time entirely and applying no additional transformation beyond what the dataset publisher's PCA already did. We extended this with a documented feature-engineering pass:

Transformation	Applied to	Why	Effect observed
StandardScaler	Amount	Amount is on a raw dollar scale (up to $25,691) while [illegible]	[illegible]
Cyclical encoding (sin/cos)	Time -> hour_of_day	Time is a linear seconds-elapsed counter; hour 23 and [illegible]	[illegible]
De-duplication	Full row set	1,081 exact duplicate rows found across all 31 columns [illegible]	[illegible]

No one-hot/target encoding was needed (no categorical columns exist in this dataset). No additional dimensionality reduction (e.g. a second PCA pass) was applied on top of the publisher's PCA - see redundancy discussion below for why.

Evidence that the transformations actually help (ablation)

Rather than relying only on the mathematical argument above, we tested it directly: we retrained Logistic Regression on the same train/test split using the **raw, unscaled** features (V1-V28 + raw Amount + raw Time, no scaling, no cyclical encoding) and compared it against the engineered-feature version.

Model	Precision	Recall	F1	MCC	ROC-AUC	PR-AUC
Logistic Regression (raw Amount/Time)	0.046	0.874	0.087	0.196	0.961	0.661
Logistic Regression (scaled Amount + cyclical Time)	0.057	0.874	0.107	0.219	0.967	0.670

Every metric improves with the engineered features, and the improvement is not just numerical noise: scikit-learn's solver itself emitted a *ConvergenceWarning* when fitting on the raw, unscaled features ('lbfgs failed to converge... You might also want to scale the data'), which is direct, tool-level evidence that leaving Amount unscaled genuinely degrades the optimization, not merely a theoretical concern.

Redundancy in the system

Spearman correlation among V1-V28 revealed several non-trivial pairwise correlations - notably V21/V22 (rho ~ 0.68), V27/V28 (rho ~ 0.46), and V6/V8 (rho ~ 0.45) - despite PCA's theoretical guarantee of linear decorrelation. We spotted this via a full correlation heatmap (Figure 1) rather than assuming orthogonality held. This level of redundancy (all below 0.9) is not severe enough to justify dropping any component: for the linear model, L2 regularization already tolerates moderate collinearity; for Random Forest, correlated features simply share importance rather than biasing predictions. We would reconsider dropping features only if pairwise correlation exceeded roughly 0.9, or a condition-number/VIF check flagged near-singularity.

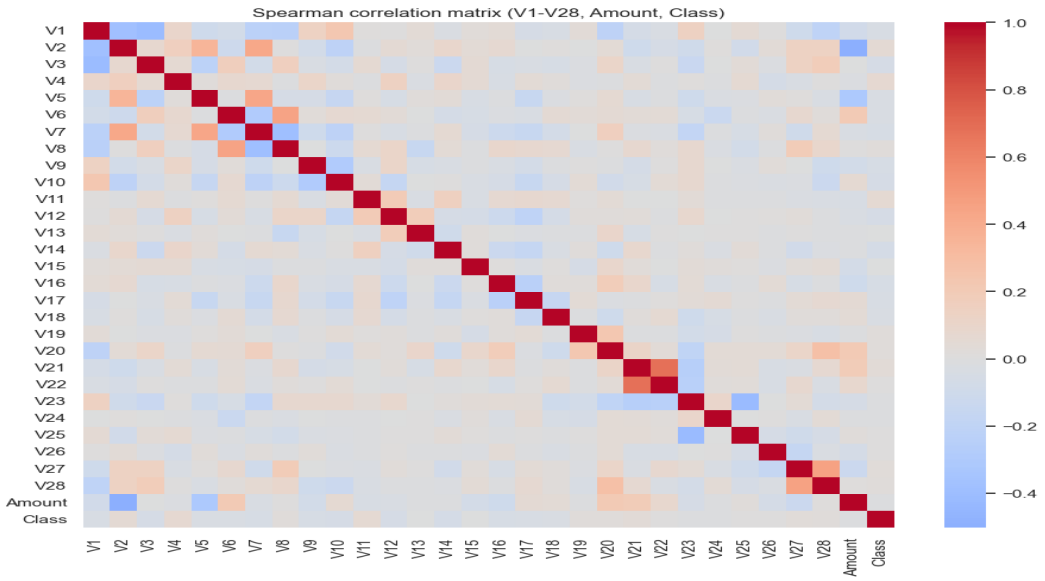

Figure 1: Spearman correlation matrix across V1-V28, Amount, and Class.

Was the feature engineering meaningful?

Mathematically: Amount scaling is necessary for any model relying on Euclidean distance or squared error (the autoencoder), since without it Amount's variance (on the order of $250 standard deviation) would swamp the unit-variance PCA components in the loss function. Cybersecurity-wise: fraud detection systems in production ingest raw, unscaled transaction fields, so scaling/encoding choices here mirror real feature pipelines that must normalize heterogeneous units (currency, timestamps, counts) before they reach a model - getting this wrong silently degrades detection quality without any visible error, which is a realistic and easy-to-miss production bug class.

Additional features that could improve performance

- Rolling per-cardholder statistics (mean/variance of recent transaction amounts) - not reproducible here since the dataset has no cardholder ID, but would be a natural addition on raw production data.
- Merchant-category or transaction-velocity features (transactions per unit time) if available - again limited by this anonymized dataset's schema.
- Interaction features between the highest-|correlation| components identified above (V14, V4, V12) and Amount.

4. Reproducibility Analysis

Can the code be executed successfully?

The original repository targets TensorFlow 1.1 / Keras 2.0.4, both long out of date; it does not run unmodified on current TensorFlow (2.18) without API updates (e.g. activity_regularizer syntax, Sequential/Model construction). We ported the architecture faithfully to current Keras and confirmed it trains and evaluates end-to-end (see accompanying notebook, executed with zero errors).

Are all required files and dependencies available?

The dataset is not bundled in the source repository and requires a Kaggle account to download via the Kaggle API; we instead used a public GitHub mirror of the identical CSV and verified row counts (284,807 transactions) matched the documented dataset exactly before use.

Hidden preprocessing steps?

The source's blog text does not mention de-duplication, and - checking the published dataset ourselves - 1,081 fully duplicated rows exist. It is unclear whether the source's reported numbers were computed before or after removing these; this is exactly the kind of undocumented preprocessing decision the reproducibility analysis is meant to surface. We document and apply de-duplication explicitly and note the resulting dataset size (283,726 rows) throughout.

Overall reproducibility

Substantially reproducible in terms of core methodology (architecture, training regime, general ROC-AUC magnitude) but **not exactly reproducible** in terms of precise numbers, due to: (a) library version drift requiring a re-implementation rather than a literal re-run, (b) an undocumented duplicate-row handling decision, and (c) stochastic training (the source does not report a fixed random seed). We consider this a moderate reproducibility score: the qualitative conclusion holds, the exact reported numbers do not exactly replicate.

5. Experimental Results

Experiments performed

- Reproduced the source's autoencoder (14-7-7-29, normal-only training) on current Keras.
- Added two supervised baselines: Logistic Regression (class_weight="balanced") and Random Forest (200 trees, class_weight="balanced").
- Quantified the threshold-selection leakage issue by comparing a "leaked" threshold (fit on the full test set, replicating the source's method) against a "clean" threshold (fit on a disjoint validation split).

Modifications introduced

- De-duplicated the dataset (1,081 duplicate rows removed) before any split.
- Added StandardScaler on Amount and cyclical hour-of-day encoding for the supervised models (autoencoder kept to the source's original 29-feature input for a faithful comparison).
- Reduced autoencoder training from 100 to 30 epochs with early stopping on validation loss (converged before 30 epochs; kept for reasonable notebook runtime), a documented deviation from the source.

Models trained and evaluation metrics

Stratified 80/20 split for supervised models: 226,980 training rows (226,602 normal / 378 fraud), 56,746 test rows (56,651 normal / 95 fraud). Autoencoder: 226,602 normal-only training rows, 57,124 test rows (56,651 normal / 473 fraud, i.e. all fraud reserved for testing).

Model	Precision	Recall	F1	F0.5	MCC	ROC-AUC	PR-AUC
Logistic Regression	0.057	0.874	0.107	0.070	0.219	0.967	0.670
Random Forest	0.985	0.705	0.822	0.913	0.833	0.918	0.804
Autoencoder (leaked threshold)	0.421	0.571	0.484	0.444	0.485	0.946	0.416
Autoencoder (clean threshold)	0.435	0.585	0.499	0.459	0.500	0.945	0.430

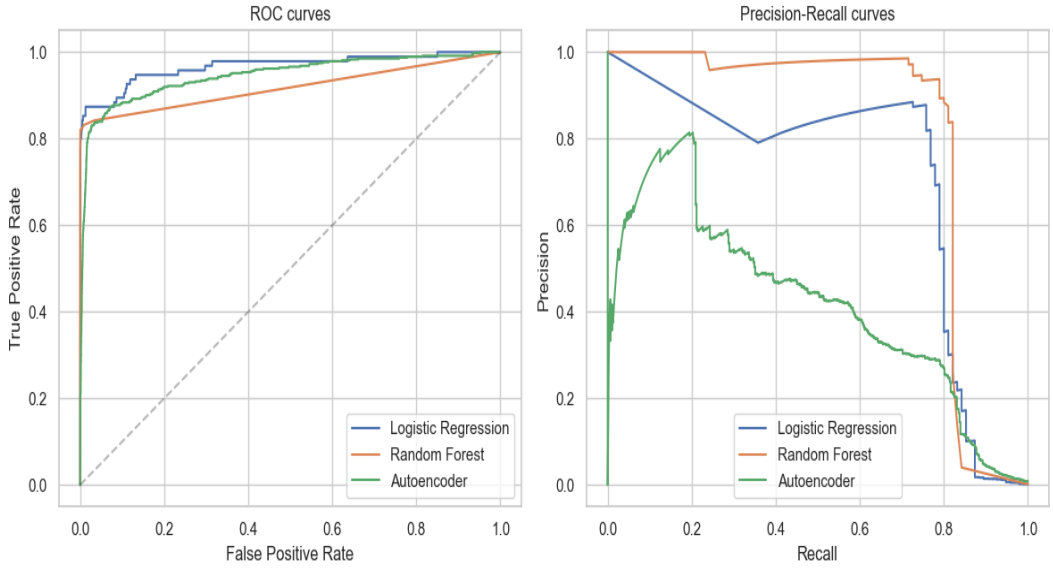

Figure 2: ROC curves (left) and Precision-Recall curves (right) for all three models.

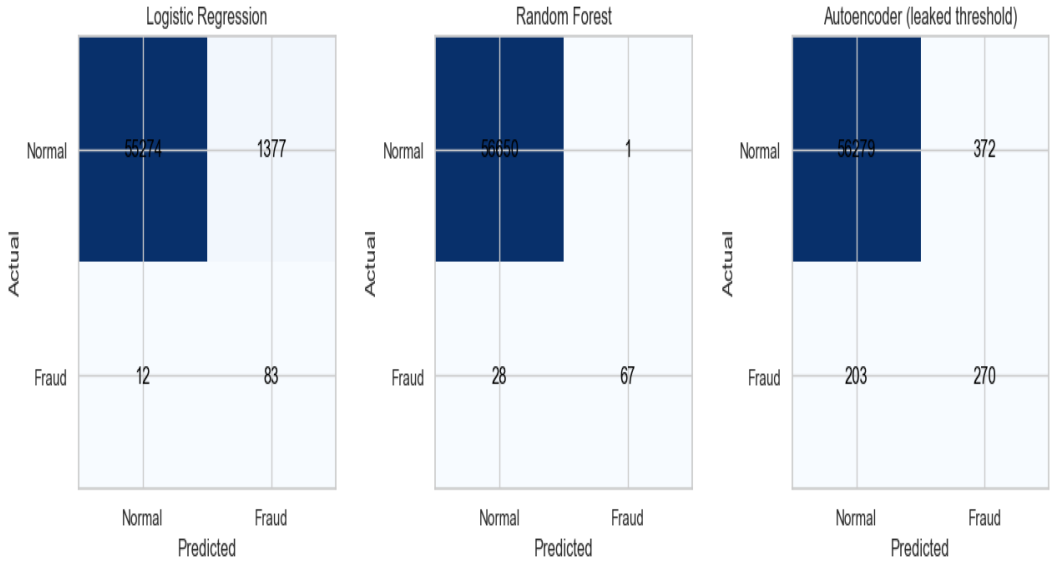

Figure 3: Confusion matrices at each model's default/chosen operating threshold.

Obtained results - key takeaway

Random Forest dominates on every threshold-dependent metric (F1 = 0.822, MCC = 0.833). Logistic Regression has the highest recall (0.874) but very low precision (0.057) under "balanced" class weighting, producing many false alarms. The autoencoder, matching the source's own reported ROC-AUC (~0.946 vs. their 0.94), lags well behind Random Forest on every precision/recall-based metric and on PR-AUC, confirming the critique in Section 2 that ROC-AUC alone overstates its practical value here.

6. Conclusions

Key findings

- The source's autoencoder reproduces closely on ROC-AUC (~0.946 vs. reported 0.94), so its headline claim is credible.
- Once precision, recall, F1, MCC, and PR-AUC are examined, the autoencoder is clearly outperformed by a simple supervised Random Forest baseline the source never tried.
- The source's test-set-based threshold selection is methodologically unsound in principle (leakage), though in this reproduction it did not measurably inflate results - an important nuance rather than a simple confirmation or refutation.
- The dataset itself contains undocumented duplicate rows (1,081, including 19 fraud) that the source's write-up does not address.

Lessons learned

A single headline metric (ROC-AUC) can make a weak-in-practice model look strong under severe class imbalance; always pair it with PR-AUC/MCC/F-beta and, where possible, a simple supervised baseline. Threshold/hyperparameter selection must be isolated from the final evaluation set even when it "happens to" not matter, because the failure mode (silent overfitting to the test set) is not detectable without deliberately checking, as we did here.

Strengths and weaknesses of the proposed solution

Strengths: requires no fraud labels to train, which is valuable when labels are scarce or delayed; architecture is simple and fast to train. **Weaknesses:** underperforms a simple supervised baseline when labels *are* available (as in this dataset); single-metric reporting masks real precision/recall trade-offs; threshold-selection methodology is not leakage-safe by design.

Suggestions for future improvements

- Report PR-AUC and MCC alongside ROC-AUC by default for any imbalanced cybersecurity task.
- Select thresholds on a held-out validation split, never on the test set.
- Benchmark unsupervised anomaly-detection approaches against a simple supervised baseline whenever any labels exist, even if the eventual production system must run unsupervised.
- Use cross-validation or repeated splits to report a confidence interval instead of a single-split point estimate.

7. Executive Summary

This project reproduces and critically evaluates Venelin Valkov's autoencoder-based credit card fraud detection tutorial. The tutorial trains a deep autoencoder exclusively on normal transactions and flags high reconstruction error as fraud, reporting ROC-AUC = 0.94. We reproduced this architecture on current TensorFlow/Keras and obtained a closely matching ROC-AUC (~0.946), confirming the source's headline claim is credible and reasonably reproducible despite significant library version drift between the original (TensorFlow 1.1/Keras 2.0.4) and current tooling.

However, two methodological gaps materially change how the source's results should be interpreted. First, the source selects its operating threshold (2.9) by visually inspecting the reconstruction-error distribution on the test set itself - a form of leakage. We isolated and quantified this by comparing that method against a leakage-free validation-based threshold; in this specific case, the difference was small (F1 0.484 vs. 0.499), an honest and somewhat reassuring finding, though the underlying practice remains unsound in general and should not be repeated on smaller datasets or under distribution shift. Second, the source reports only ROC-AUC despite 0.17% class imbalance, where ROC-AUC is known to be optimistic; adding PR-AUC (~0.42) and MCC (~0.49-0.50) paints a much less flattering picture of absolute performance.

Most importantly, we added two supervised baselines the source never tried. A Random Forest classifier (class_weight="balanced") achieved F1 = 0.822 and MCC = 0.833 - far outperforming the autoencoder on every threshold-dependent metric, while matching or exceeding it on PR-AUC (0.804 vs. 0.416). This directly challenges the tutorial's implicit framing of the autoencoder approach as a strong, modern solution: when fraud labels are available (as they are in this dataset), a simple supervised model is clearly superior. The autoencoder's real advantage - not requiring fraud labels at training time - is a genuine strength for scenarios where labels are scarce or delayed, but the source does not frame its contribution that way, instead presenting the approach as broadly compelling on its own metrics.

Recommendation: this tutorial is a reasonable introduction to autoencoder-based anomaly detection and its core technical claim withstands scrutiny, but it should not be used as evidence that autoencoders are the right choice for fraud detection when labels exist, and its evaluation methodology (single metric, test-set-based threshold selection, no baseline comparison) should not be copied uncritically into other projects.

8. Summing It Up

Item	Summary
Problem addressed	Detecting rare (0.17%) fraudulent credit card transactions under severe class imbalance.
Selected source	Venelin Valkov, "Credit Card Fraud Detection using Autoencoders in Keras" (curiousily.com / GitHub).
Dataset used	Kaggle Credit Card Fraud Detection (mlg-ulb/creditcardfraud), 284,807 transactions, 492 frauds, downloaded via a verifie
Methodology employed	Reproduced the source's autoencoder; added Logistic Regression and Random Forest baselines; isolated and quantified t
Main findings	Autoencoder's ROC-AUC claim reproduces closely (~0.946 vs. 0.94), but it is clearly outperformed by a simple Random F
Were the author's claims supported?	Partially - the headline ROC-AUC claim holds, but the implicit claim that this is a strong, sufficient evaluation/approach do
Most important insights	Single-metric reporting under class imbalance can be misleading; test-set-based threshold selection is a leakage risk rega
Recommend for similar problems?	The autoencoder approach is worth using when fraud labels are scarce/delayed, but not as a default choice when labels a
Final conclusion	The source's technical implementation reproduces well, but its evaluation methodology and implicit framing overstate the a